

Prompt Injection and Tool-Use Security in LLM Agents

Abstract

Large language model agents have rapidly evolved from conversational interfaces into systems that browse the web, call APIs, execute code, manipulate files, maintain memory, and coordinate with other agents. This expansion of capability has created a corresponding expansion of attack surface: malicious instructions can now arrive not only from users, but also from retrieved documents, tool outputs, HTML, screenshots, long-term memory, chat templates, function metadata, and inter-agent messages. The literature has converged on prompt injection as the central agent-security problem, but the field is no longer limited to prompt strings alone. Recent work shows that agent compromise can propagate through retrieval poisoning, privilege misuse, memory persistence, multimodal environmental manipulation, protocol-layer abuse, and tool-description poisoning, while effective defense increasingly requires runtime policy enforcement, provenance, isolation, and least-privilege rather than prompt engineering alone [1-8, 10-18, 32-43, 70-83]. ¹

This survey synthesizes the academic literature on prompt injection and tool-use security in LLM agents. It first situates the problem within the broader rise of tool-enabled agents; then organizes research into attack families, defense families, and evaluation methodologies; and finally identifies open problems around contextual security, compositionality, long-horizon state, multimodality, privacy, and realistic benchmarking. A central conclusion of the literature is that prompt injection is not merely a prompting flaw but a systems-security problem caused by the collapse of traditional boundaries between code and data, authority and observation, intention and execution. Accordingly, the strongest recent defenses increasingly borrow from classical security ideas—control-flow integrity, access control, information-flow constraints, reference monitoring, and contextual integrity—while adapting them to stochastic, natural-language-driven agent pipelines [6-8, 17-25, 70-83]. ²

Introduction

The modern LLM agent stack was enabled by a sequence of advances in tool use and environment interaction. Toolformer showed that language models can learn when and how to invoke tools; ReAct made interleaved reasoning and action a general prompting pattern; Gorilla, API-Bank, ToolLLM, ToolSandbox, ToolTalk, MetaTool, and τ -bench converted tool calling into a trainable and benchmarkable capability; and WebArena, VisualWebArena, and OSWorld moved the field from static question answering toward open-ended web and computer control [2, 5, 84-96]. These systems are powerful precisely because they externalize cognition into tool calls, retrieval, and actuation—but that same externalization makes them security-critical. Once an agent can send messages, execute commands, transfer money, access files, or browse on a user's behalf, even subtle control failures can cause material harm. ³

Prompt injection emerged as the canonical articulation of this risk. Early work such as Ignore Previous Prompt and Prompt Injection Attack against LLM-Integrated Applications demonstrated that instruction-following models can be induced to ignore developer intent, leak hidden prompts, or pursue attacker-specified goals [9, 10]. Follow-on work formalized attack goals and built controlled evaluation suites, while BIPIA, instruction-following robustness benchmarks, and prompt-extraction studies showed that

the problem was neither anecdotal nor limited to a single product class [11-17]. A particularly influential diagnosis is that current LLMs do not natively enforce a principled separation between instructions and data. When agents ingest untrusted content, they often lack a reliable mechanism for deciding what should be executed as an imperative and what should be treated as passive information [12, 17]. 4

In agents, that difficulty compounds. Indirect prompt injection literature shows that the malicious instruction is often not typed by the attacker into the front-end at all; instead, it is smuggled through retrieved email, documents, search results, webpages, calendars, browser DOMs, or MCP tool metadata. Multi-agent work further shows that one compromised agent can infect downstream agents. Memory-poisoning work extends the attack horizon across sessions, and multimodal work shows that screenshots, visual overlays, or environmental perturbations can redirect browser and computer-use agents. Privacy and privilege papers then add a second systems dimension: even if an agent completes the “right” task, it may still call the wrong tool, disclose more data than necessary, or act with unjustified authority [32-43, 47-69, 97-100]. 5

The academic literature has therefore moved from a narrow notion of “bad prompts” to a broader theory of contextual security for agentic systems. The core question is no longer merely whether an LLM can refuse a malicious sentence, but whether an agent can preserve task alignment, source authorization, action alignment, and information-flow integrity across a long, stateful, tool-mediated trajectory [6-8, 41, 70-83]. That reframing is the basis for the method families reviewed next. 6

Methods

A useful synthesis of the literature is to organize methods by where control is gained or defended in the agent pipeline: at the prompt boundary, across retrieved or tool-returned context, inside memory and state, across multimodal perception, between agents, and at the eventual action boundary.

At the **prompt and context layer**, the field distinguishes direct prompt injection, indirect prompt injection, jailbreak-style role or policy bypass, and prompt extraction. Direct prompt injection asks the model to ignore prior instructions or reveal secrets [9-16, 29-31]. Indirect prompt injection embeds those directives in external content, producing attacks that are more realistic for deployed agents because they exploit ordinary retrieval and tool use [12, 17, 32, 33, 41-45]. BIPIA and InjecAgent established the first benchmarked settings for such attacks, while AgentDojo and ASB escalated the evaluation from text outputs to realistic tool trajectories [12, 32-34]. More recent work demonstrates that attack effectiveness is highly sensitive to context: LLMail-Inject gathered over two hundred thousand adaptive attack submissions against email assistants; AgentPI and CAPTURE showed that defenses that appear strong on static benchmarks often overfit to narrow interaction regimes; and empirical “in-the-wild” studies stress retrieval realism as a missing variable in many earlier evaluations [28, 41, 42, 64]. 7

At the **automated attack-construction layer**, the literature increasingly replaces handwritten strings with optimization, search, or adaptive feedback. The automatic and universal prompt-injection work of Liu et al. showed that prompt injection can be cast as a goal-directed optimization problem rather than artisanal prompt crafting [14]. Adaptive evaluation later demonstrated that many defenses collapse once the adversary optimizes specifically against them, a theme reinforced by challenge datasets, public attack competitions, and subsequent automated attack frameworks [16, 42, 76]. ChatInject exploits structured chat templates and persuasive multi-turn dialogues, showing that role tokens and formatting conventions can themselves become an attack surface [45]. ToolHijacker extends the problem from context strings to the planner’s tool-selection process, poisoning tool documentation so that the agent chooses attacker-favorable tools [50]. Imprompter and related work similarly show that obfuscated or optimized prompts can induce improper tool use even when they are not obviously malicious to a human

reader [51]. Collectively, these papers shift the field from “can an agent be tricked?” to “how cheaply and generally can adversarial control be synthesized?” 8

At the **stateful and systems attack layer**, the literature broadens further. Multi-agent studies such as Prompt Infection and MAS control-flow hijacking show that prompt injection can self-propagate across communication channels, effectively turning natural-language messages into a worm-like control vector [47, 48]. Backdoor work—including BadAgent, early backdoor threat studies, AgentPoison, BackdoorAgent, data-exfiltration attacks through backdoored tool use, and later defensive work such as ReAgent—demonstrates that compromise can be implanted during fine-tuning or memory construction and then activated later under semantic triggers [52-60]. Memory-specific papers argue that persistent memory is a qualitatively new attack surface because poisoning today can influence tool choice or actuation across future sessions, with persistence, propagation, and rollback becoming first-class security concerns [55-57]. RAG and corpus-poisoning papers extend this logic to retrieval pipelines, where a poisoned document can be selected, summarized, and operationalized by downstream action logic [54, 65-68]. These attacks are important because they reveal that prompt injection is often only the visible symptom of a deeper problem: the agent is executing attacker-controlled state transitions. 9

At the **multimodal and environmental layer**, browser and computer-use agents expose vulnerabilities unavailable to text-only systems. Visual Prompt Injection Attacks for Computer-Use Agents introduced VPI-Bench and demonstrated that malicious visual instructions embedded in webpages can steer agents into attacker-chosen actions [61]. EnvInjection and related web-agent environment attacks optimize perturbations in rendered webpages, while pop-up-based attacks show that even simple distractions can redirect multimodal agents [62]. Context-manipulation work then identifies “plan injection,” in which the adversary corrupts the agent’s internal task representation rather than merely providing a conflicting instruction [63]. This is a significant conceptual advance: a secure agent must defend not only against imperative strings but against manipulations of salience, latent plan state, and visual attention. That insight is especially relevant as computer-use agents become more reliant on screenshots than structured DOM text [40, 43, 61-63, 94-96]. 10

Defenses fall into four broad families. The first is **representation and training-time hardening**, which tries to teach the model to honor privilege distinctions inside its context. Instruction Hierarchy explicitly trains models to prioritize system and developer instructions over untrusted data; StruQ separates prompts and data into structured channels; Jatmo and SecAlign construct task- or preference-optimized models that are supposed to ignore injected instructions; ASIDE moves the separation idea into the embedding architecture; and Defensive Tokens offer a lighter-weight inference-time analogue [18-25]. These methods are valuable because they target the root cause identified by instruction/data separation papers. Yet several later evaluations argue that simple separation is not sufficient in context-dependent tasks where legitimate action genuinely depends on runtime observations [17, 31, 41]. 11

The second family is **detection, sanitization, and causal diagnosis**. This includes classifier-style detection such as attention or embedding-based approaches [26, 27], guardrail-style context-aware detectors such as CAPTURE [28], and runtime sanitizers such as PromptArmor [77]. More sophisticated runtime defenses use re-execution or attribution: MELON compares masked re-executions; DRIFT dynamically plans and validates minimal trajectories while isolating memory streams; CausalArmor and AttriGuard ask why a privileged action was produced and trigger sanitization only when untrusted context has dominant causal influence; AgentSentry models indirect prompt injection as temporal causal takeover across tool-return boundaries [75, 78-81]. These methods are closely related conceptually even when their machinery differs: all of them treat security as a question of causal dependence between the user’s authorized task and the action the agent is about to take. 12

The third family is **security-by-design execution architectures**. CaMeL is the most explicit example, extracting trusted control flow and constraining untrusted data so that it cannot directly alter program flow [70]. Design-pattern work generalizes this idea into architectural templates with provable or near-provable resistance properties [71]. Progent, Prompt Flow Integrity, and AgentGuard then move toward privilege control and reference monitoring, enforcing least privilege for tool invocation and checking data-flow constraints rather than trusting the model to reason correctly every time [72, 73, 83]. MCIP transports similar ideas to the Model Context Protocol ecosystem, arguing that protocol standardization without contextual integrity merely enlarges the attack surface [82]. These papers collectively represent the most security-native trend in the literature: instead of “asking the model nicely” not to fail, they wrap the model in monitors, policies, and capability boundaries. ¹³

The fourth family is **benchmarking and evaluation methodology**, which has become a research area of its own. Security papers increasingly report not just attack success rate, but also task success, utility under attack, false positives, latency, over-defense, privacy leakage, privilege misuse, and trajectory-level policy violations. BIPIA and early robustness studies are mainly text-centric [12, 13]; AgentDojo, ASB, Agent-SafetyBench, ToolEmu, AgentHarm, OpenAgentSafety, OS-Harm, and WASP move evaluation into tool-mediated, interactive, and multimodal environments [33-40, 43]. ToolPrivacyBench and AgentLeak show that output-only evaluation misses internal-channel leakage through tool arguments and inter-agent messages [97, 98]. The field now widely acknowledges a three-way tradeoff among security, utility, and cost or latency, and several recent SoK-style papers caution that weak or static benchmarks can make simple defenses appear stronger than they are under adaptive attack [41, 76]. ¹⁴

Challenges and Prospects

The most important unresolved issue is **semantic authority**. Traditional systems can often separate code from data syntactically, but LLM agents interpret both through the same generative interface. Instruction Hierarchy, StruQ, SecAlign, and ASIDE all improve this situation, yet a recurring finding is that purely representational defenses can become brittle when the user’s legitimate objective depends on subtle environmental context or when the attacker manipulates the framing of that context rather than inserting an obviously imperative command [17-25, 31, 41]. The strongest prospect in the literature is therefore not one universal prompt filter, but a layered notion of intent-to-execution integrity: provenance for untrusted content, explicit authority types for tools and memories, deterministic enforcement at action boundaries, and causal tests that verify whether an upcoming action is still justified by the user’s task [70-83]. ¹⁵

A second challenge is **time and persistence**. Stateless chatbot safety does not transfer cleanly to agents that write memory, revisit state, or coordinate over long horizons. Memory-poisoning papers show that the attack may not be visible when it is planted, only when it is executed later [55-57]. Privacy papers make the same point for disclosure: the harmful leak may occur through an internal handoff rather than a final answer [97-99]. Multi-agent security intensifies this problem because intermediate messages, shared plans, and hidden coordinator channels become additional trust boundaries [47, 48, 98]. Future work will likely need lifecycle governance for agent memory—write-time provenance, versioning, selective revocation, rollback, forgetting, and auditability—rather than retrieval-time moderation alone [56, 57, 97, 98]. ¹⁶

A third challenge is **realism of evaluation**. The literature increasingly agrees that static strings, toy tools, and oversimplified success metrics underestimate both attackers and defenders. LLMail-Inject, adaptive-attack papers, AgentPI, WASP, and recent browser-agent studies all show that benchmark conclusions change materially when attacks are optimized to the defense, when context dependence is preserved, or when the environment contains realistic distractors and action opportunities [41-45, 61-63, 76]. This has

two implications. First, evaluation should be trajectory-based and deployment-aware, measuring what information was observed, trusted, transmitted, and acted upon—not only whether the final answer looked acceptable. Second, benchmark governance itself is becoming an open problem: several recent papers explicitly argue that saturated or flawed benchmarks can misdirect the field by rewarding over-defense, brittle filtering, or attack sets that do not transfer in practice [41, 76]. ¹⁷

A final prospect is **convergence with classical security architecture**. The best recent work increasingly uses concepts familiar from systems security—least privilege, capability security, reference monitors, information-flow control, contextual integrity, sandboxing, same-origin restrictions, and protocol hardening—but reinterprets them for language-based control [70-83]. The likely long-term outcome is not a single monolithic “safe agent model,” but a stack consisting of a competent planner, typed and minimized tool interfaces, security labels on observations and memories, deterministic policy enforcement on high-impact actions, and benchmark suites that measure both usefulness and policy compliance in realistic environments. The literature does not yet have a universal design, but it increasingly agrees on the needed ingredients. ¹⁸

Conclusion

The research literature now makes three points with unusual consistency. First, prompt injection is the defining vulnerability of tool-using LLM agents, but it is better understood as a systems problem than a prompt-engineering problem. Second, merely improving refusal at the model-output layer is insufficient once authority is mediated through retrieval, tools, memory, GUI perception, or other agents. Third, the most promising defenses are layered architectures that combine model hardening with provenance, privilege control, causal or counterfactual checks, and deterministic runtime enforcement [17-25, 41, 70-83]. ¹⁹

The field remains immature. Benchmarks are improving but still uneven; adaptive attacks still overpower many published defenses; and long-term memory, privacy-preserving tool use, protocol-level interoperability, and multimodal perception remain under-defended. Even so, the literature has progressed from anecdotal exploit demonstrations to a serious security discipline for agentic AI. The central research agenda now is clear: secure the transition from user intent to external action under uncertainty, untrusted context, and long-horizon state. That agenda is difficult, but it is also increasingly well specified by the current academic record [6-8, 39-43, 76, 97-101]. ²⁰

References

- [1] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, et al. A Survey of Large Language Models. arXiv:2303.18223, 2023.
- [2] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool Learning with Large Language Models: A Survey. arXiv:2405.17935, 2024.
- [3] Yihe He, Shitong Zhang, Han Liu, and others. The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies. arXiv:2407.19354, 2024.
- [4] Yisong Wang, Yiran Yang, et al. A Comprehensive Survey in LLM(-Agent) Full Stack Safety. arXiv: 2504.15585, 2025.
- [5] Junyu Luo, Weizhi Zhang, Ye Yuan, et al. Large Language Model Agent: A Survey on Methodology, Applications and Challenges. arXiv:2503.21460, 2025.

- [6] Zhenyu Chu, et al. A Systematic Survey of Security Threats and Defenses in LLM-Based AI Agents: A Layered Attack Surface Framework. arXiv:2604.23338, 2026.
- [7] Yifei Wang, et al. The Attack and Defense Landscape of Agentic AI. arXiv:2603.11088, 2026.
- [8] Anonymous. Toward Secure LLM Agents: Threat Surfaces, Attacks, Defenses, and Evaluation. arXiv:2606.10749, 2026.
- [9] Fábio Perez and Ian Ribeiro. Ignore Previous Prompt: Attack Techniques for Language Models. NeurIPS ML Safety Workshop, 2022. arXiv:2211.09527.
- [10] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt Injection Attack against LLM-Integrated Applications. arXiv:2306.05499, 2023.
- [11] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. USENIX Security, 2024. arXiv:2310.12815.
- [12] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. KDD, 2025. arXiv:2312.14197.
- [13] Zekun Li, Baolin Peng, Pengcheng He, and Xifeng Yan. Evaluating the Instruction-Following Robustness of Large Language Models to Prompt Injection. EMNLP, 2024.
- [14] Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. Automatic and Universal Prompt Injection Attacks against Large Language Models. arXiv:2403.04957, 2024.
- [15] Yiming Zhang and Daphne Ippolito. Effective Prompt Extraction from Language Models. CoLM, 2024. arXiv:2307.06865.
- [16] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, Eric Wong, and Asher Grattafiori. Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs through a Global Prompt Hacking Competition. arXiv:2311.16119, 2023.
- [17] Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert. Can LLMs Separate Instructions From Data? And What Do We Even Mean By That? arXiv:2403.06833, 2024.
- [18] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208, 2024.
- [19] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending Against Prompt Injection with Structured Queries. USENIX Security, 2025. arXiv:2402.06363.
- [20] Julien Piet, Maha Alrashed, Chawin Sitawarin, Sizhe Chen, Zeming Wei, Elizabeth Sun, Basel Alomair, and David Wagner. Jatmo: Prompt Injection Defense by Task-Specific Finetuning. ESORICS, 2024. arXiv:2312.17673.
- [21] Tianhao Wu, et al. Improving LLM Safety with Instruction Hierarchy. OpenReview, 2024.

- [22] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending Against Prompt Injection with Preference Optimization. CCS, 2025. arXiv:2410.05451.
- [23] Egor Zverev, Sahar Abdelnabi, et al. ASIDE: Architectural Separation of Instructions and Data in Language Models. arXiv:2503.10566, 2025.
- [24] Chenchen Guo, et al. IH-Challenge: A Training Dataset to Improve Instruction-Hierarchy Robustness. arXiv:2603.10521, 2026.
- [25] Sizhe Chen, et al. Defending Against Prompt Injection With a Few Defensive Tokens. arXiv:2507.07974, 2025.
- [26] Kai-Hung Hung, et al. Attention Tracker: Detecting Prompt Injection Attacks in LLMs. Findings of NAACL, 2025.
- [27] Md Ahsan Ayub and Subhabrata Majumdar. Embedding-Based Classifiers Can Detect Prompt Injection Attacks. arXiv:2410.22284, 2024.
- [28] Gaurav Kholkar, et al. CAPTURE: Context-Aware Prompt Injection Testing and Robustness Evaluation. arXiv:2505.12368, 2025.
- [29] Jiahao Yu, Yuhang Wu, Dong Shu, Mingyu Jin, Sabrina Yang, and Xinyu Xing. Assessing Prompt Injection Risks in 200+ Custom GPTs. arXiv:2311.11538, 2023.
- [30] Badhan Chandra Das, M. Hadi Amini, and Yanzhao Wu. System Prompt Extraction Attacks and Defenses in Large Language Models. arXiv:2505.23817, 2025.
- [31] Zhixiong Zhuang, Maria-Irina Nicolae, Hui-Po Wang, and Mario Fritz. ProxyPrompt: Securing System Prompts against Prompt Extraction Attacks. arXiv:2505.11459, 2025.
- [32] Qiusi Zhan, Zhenghao Liang, Yihan Ying, and Daniel Kang. Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. Findings of ACL, 2024. arXiv:2403.02691.
- [33] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS Datasets and Benchmarks, 2024. arXiv:2406.13352.
- [34] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent Security Bench: Formalizing and Benchmarking Attacks and Defenses in LLM-Based Agents. ICLR, 2025. arXiv:2410.02644.
- [35] Zhaolin Zhang, et al. Agent-SafetyBench: Evaluating the Safety of LLM Agents. arXiv:2412.14470, 2024.
- [36] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. arXiv:2401.10019, 2024.

- [37] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. ICLR, 2024. arXiv:2309.15817.
- [38] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. ICLR, 2025. arXiv:2410.09024.
- [39] Sanidhya Vijayvargiya, Aditya Bharat Soni, Xuhui Zhou, Zora Zhiruo Wang, Nouha Dziri, Graham Neubig, and Maarten Sap. OpenAgentSafety: A Comprehensive Framework for Evaluating Real-World AI Agent Safety. ICLR, 2026. arXiv:2507.06134.
- [40] Thomas Kuntz, Agatha Duzan, Hao Zhao, Francesco Croce, Zico Kolter, Nicolas Flammarion, and Maksym Andriushchenko. OS-Harm: A Benchmark for Measuring Safety of Computer Use Agents. arXiv:2506.14866, 2025.
- [41] Peiran Wang, Xinfeng Li, Chong Xiang, Jinghui Zhang, Ying Li, Lixia Zhang, Xiaofeng Wang, and Yuan Tian. The Landscape of Prompt Injection Threats in LLM Agents: From Taxonomy to Analysis. arXiv:2602.10453, 2026.
- [42] Sahar Abdelnabi, Aideen Fay, Ahmed Salem, Egor Zverev, Kai-Chieh Liao, Chi-Huang Liu, Chun-Chih Kuo, Jannis Weigend, Danyael Manlangit, Alex Apostolov, et al. LLMail-Inject: A Dataset from a Realistic Adaptive Prompt Injection Challenge. arXiv:2506.09956, 2025.
- [43] Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks. arXiv:2504.18575, 2025.
- [44] Kaiyuan Zhang, Mark Tenenholtz, Kyle Polley, Jerry Ma, Denis Yarats, and Ninghui Li. BrowseSafe: Understanding and Preventing Prompt Injection Within AI Browser Agents. arXiv:2511.20597, 2025.
- [45] Hwan Chang, Yonghyun Jun, and Hwanhee Lee. ChatInject: Abusing Chat Templates for Prompt Injection in LLM Agents. arXiv:2509.22830, 2025.
- [46] Anonymous. Phantom: Automating Agent Hijacking via Structural Template Injection. arXiv:2602.16958, 2026.
- [47] Donghyun Lee and Mo Tiwari. Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems. arXiv:2410.07283, 2024.
- [48] Harlan Triedman, et al. Multi-Agent Systems Execute Arbitrary Malicious Code. arXiv:2503.12188, 2025.
- [49] Yuxiang Hu, et al. Log-To-Leak: Prompt Injection Attacks on Tool-Using LLM Agents. OpenReview, 2026.
- [50] Anonymous. Prompt Injection Attack to Tool Selection in LLM Agents. arXiv:2504.19793, 2025.
- [51] Anonymous. Imprompter: Tricking LLM Agents into Improper Tool Use. arXiv:2410.14923, 2024.

- [52] Yuhan Wang, et al. Inserting and Activating Backdoor Attacks in LLM Agents. arXiv:2406.03007, 2024.
- [53] Wei Yang, et al. Watch Out for Your Agents! Investigating Backdoor Threats on LLM-Based Agents. arXiv:2402.11208, 2024.
- [54] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-Teaming LLM Agents via Poisoning Memory or Knowledge Bases. arXiv:2407.12784, 2024.
- [55] B. D. Sunil, et al. Memory Poisoning Attack and Defense on LLM Agents. arXiv:2601.05504, 2026.
- [56] Yannik Louck, et al. Securing LLM-Agent Long-Term Memory Against Poisoning. arXiv:2606.24322, 2026.
- [57] Zehao Lin, Xixuan Hao, Renyu Fu, Shaobo Cui, Kai Chen, Chunyu Li, Zhiyu Li, and Feiyu Xiong. A Survey on Long-Term Memory Security in LLM Agents: Attacks, Defenses, and Governance Across the Memory Lifecycle. arXiv:2604.16548, 2026.
- [58] Yunhao Feng, Yige Li, Yutao Wu, Yingshui Tan, Yanming Guo, Yifan Ding, Kun Zhai, Xingjun Ma, and Yugang Jiang. BackdoorAgent: A Unified Framework for Backdoor Attacks on LLM-Based Agents. arXiv: 2601.04566, 2026.
- [59] Wenjing Zhang, et al. Data Exfiltration via Backdoored Tool Use. arXiv:2604.05432, 2026.
- [60] Anonymous. Your Agent Can Defend Itself against Backdoor Attacks. arXiv:2506.08336, 2025.
- [61] Tiange Cao, et al. Visual Prompt Injection Attacks for Computer-Use Agents. arXiv:2506.02456, 2025.
- [62] Xilong Wang, John Bloch, Zedian Shao, Yuepeng Hu, Shuyan Zhou, and Neil Zhenqiang Gong. EnvInjection: Environmental Prompt Injection Attack to Multi-Modal Web Agents. arXiv:2505.11717, 2025.
- [63] A. S. Patlan, et al. Context Manipulation Attacks: Web Agents Are Susceptible to Indirect Prompt Injection and Memory Manipulation. arXiv:2506.17318, 2025.
- [64] Hwan Chang, et al. Indirect Prompt Injection in the Wild for LLM Systems. USENIX Security, 2026. arXiv:2601.07072.
- [65] Hongru Song, Yu-An Liu, Ruqing Zhang, Jiafeng Guo, Maarten de Rijke, Yixing Fan, and Xueqi Cheng. AdversarialCoT: Single-Document Retrieval Poisoning for LLM Reasoning. arXiv:2604.12201, 2026.
- [66] Z. Chang, et al. One Shot Dominance: Knowledge Poisoning Attack on Retrieval-Augmented Generation Systems. Findings of EMNLP, 2025.
- [67] X. Tan, et al. RevPRAG: Revealing Poisoning Attacks in Retrieval-Augmented Generation. Findings of EMNLP, 2025.
- [68] H. Wang, et al. PIDP-Attack: Combining Prompt Injection with Database Poisoning in Retrieval-Augmented Generation. arXiv:2603.25164, 2026.
- [69] Narek Maloyan, et al. Prompt Injection Attacks on Agentic Coding Assistants. arXiv:2601.17548, 2026.

- [70] Edoardo DeBenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating Prompt Injections by Design. arXiv:2503.18813, 2025.
- [71] Luca Beurer-Kellner, Marc Fischer, Daniel Das, and Franz Baader. Design Patterns for Securing LLM Agents against Prompt Injection. arXiv:2506.08837, 2025.
- [72] Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703, 2025.
- [73] Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents. arXiv:2503.15547, 2025.
- [74] Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. arXiv:2412.16682, 2024.
- [75] Kaijie Zhu, Xianjun Yang, Jindong Wang, Wenbo Guo, and William Yang Wang. MELON: Indirect Prompt Injection Defense via Masked Re-execution and Tool Comparison. arXiv:2502.05174, 2025.
- [76] Rishika Bhagwatkar, Kevin Kasa, Abhay Puri, Gabriel Huang, Irina Rish, Graham W. Taylor, Krishnamurthy Dj Dvijotham, and Alexandre Lacoste. Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks? arXiv:2510.05244, 2025.
- [77] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. PromptArmor: Simple yet Effective Prompt Injection Defenses. arXiv:2507.15219, 2025.
- [78] Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents. arXiv:2506.12104, 2025.
- [79] Minbeom Kim, Mihir Parmar, Phillip Wallis, Lesly Miculicich, Kyomin Jung, Krishnamurthy Dj Dvijotham, Long T. Le, and Tomas Pfister. CausalArmor: Efficient Indirect Prompt Injection Guardrails via Causal Attribution. arXiv:2602.07918, 2026.
- [80] Yu He, Haozhe Zhu, Yiming Li, Shuo Shao, Hongwei Yao, Zhihao Liu, and Zhan Qin. AttriGuard: Defeating Indirect Prompt Injection in LLM Agents via Causal Attribution of Tool Invocations. arXiv:2603.10749, 2026.
- [81] Tian Zhang, Yiwei Xu, Juan Wang, Keyan Guo, Xiaoyang Xu, Bowen Xiao, Quanlong Guan, Jinlin Fan, Jiawei Liu, Zhiquan Liu, and Hongxin Hu. AgentSentry: Mitigating Indirect Prompt Injection in LLM Agents via Temporal Causal Diagnostics and Context Purification. arXiv:2602.22724, 2026.
- [82] Hongtao Jing, Haoran Li, et al. MCIP: Protecting MCP Safety via Model Contextual Integrity Protocol. arXiv:2505.14590, 2025.
- [83] Anonymous. An Attribute-Based Access Control Framework for Tool-Use LLM Agents. arXiv:2605.28071, 2026.

- [84] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761, 2023.
- [85] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. arXiv:2210.03629.
- [86] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334, 2023.
- [87] Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. EMNLP, 2023. arXiv:2304.08244.
- [88] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv: 2307.16789, 2023.
- [89] Cheng-Yu Hsieh, Si-An Chen, Chun-Liang Li, Yasuhisa Fujii, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. Tool Documentation Enables Zero-Shot Tool-Usage with Large Language Models. arXiv:2308.00675, 2023.
- [90] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. arXiv:2408.04682, 2024.
- [91] Nicholas Farn and Richard Shin. ToolTalk: Evaluating Tool-Usage in a Conversational Setting. arXiv: 2311.10775, 2023.
- [92] Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao Wan, Neil Zhenqiang Gong, and Lichao Sun. MetaTool Benchmark for Large Language Models: Deciding Whether to Use Tools and Which to Use. ICLR, 2024. arXiv:2310.03128.
- [93] Shunyu Yao, et al. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Customer Service Domains. arXiv:2406.12045, 2024.
- [94] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854, 2023.
- [95] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. arXiv:2401.13649, 2024.
- [96] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. NeurIPS, 2024. arXiv:2404.07972.
- [97] Shijing Hu, Liang Liu, Zhu Meng, and Zhicheng Zhao. ToolPrivacyBench: Benchmarking Purpose-Bound Privacy in Tool-Using LLM Agents. arXiv:2606.28061, 2026.

[98] Fares E. Yagoubi, et al. AgentLeak: A Benchmark for Internal-Channel Privacy Leakage in Multi-Agent LLM Systems. arXiv:2602.11510, 2026.

[99] Arman Zharmagambetov, et al. Privacy Leakage Evaluation for Autonomous Web Agents. arXiv: 2503.09780, 2025.

[100] Itamar Levy, et al. ST-WebAgentBench: A Benchmark for Evaluating Safety and Trustworthiness in Web Agents. arXiv:2410.06703, 2024.

[101] Daoyuan Zhang, et al. MCP Security Bench: Benchmarking Attacks against Tool-Using LLM Agents in the MCP Ecosystem. arXiv:2510.15994, 2025.

[102] Yixuan Yang, Cuifeng Gao, Daoyuan Wu, Yufan Chen, Yingjiu Li, and Shuai Wang. MCPSecBench: A Systematic Security Benchmark and Playground for Testing Model Context Protocols. arXiv:2508.13220, 2025.

1 Large Language Model Agent: A Survey on Methodology, Applications and Challenges

https://arxiv.org/abs/2503.21460?utm_source=chatgpt.com

2 15 19 Can LLMs Separate Instructions From Data? And What Do We Even Mean By That?

https://arxiv.org/abs/2403.06833?utm_source=chatgpt.com

3 Toolformer: Language Models Can Teach Themselves to Use Tools

https://arxiv.org/abs/2302.04761?utm_source=chatgpt.com

4 Ignore Previous Prompt: Attack Techniques For Language Models

https://arxiv.org/abs/2211.09527?utm_source=chatgpt.com

5 Benchmarking Indirect Prompt Injections in Tool-Integrated ...

https://aclanthology.org/2024.findings-acl.624/?utm_source=chatgpt.com

6 A Framework for Formalizing LLM Agent Security

https://arxiv.org/pdf/2603.19469?utm_source=chatgpt.com

7 Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models

https://arxiv.org/abs/2312.14197?utm_source=chatgpt.com

8 Automatic and Universal Prompt Injection Attacks against ...

https://arxiv.org/abs/2403.04957?utm_source=chatgpt.com

9 Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems

https://arxiv.org/abs/2410.07283?utm_source=chatgpt.com

10 Visual Prompt Injection Attacks for Computer-Use Agents

https://arxiv.org/html/2506.02456v2?utm_source=chatgpt.com

11 The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions

https://arxiv.org/abs/2404.13208?utm_source=chatgpt.com

12 Attention Tracker: Detecting Prompt Injection Attacks in LLMs

https://aclanthology.org/2025.findings-naacl.123.pdf?utm_source=chatgpt.com

13 18 Defeating Prompt Injections by Design

https://arxiv.org/abs/2503.18813?utm_source=chatgpt.com

14 AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents

https://arxiv.org/abs/2406.13352?utm_source=chatgpt.com

16 [2601.05504] Memory Poisoning Attack and Defense on ...

https://arxiv.org/abs/2601.05504?utm_source=chatgpt.com

17 LLMail-Inject: A Dataset from a Realistic Adaptive Prompt Injection Challenge

https://arxiv.org/abs/2506.09956?utm_source=chatgpt.com

20 A Systematic Survey of Security Threats and Defenses in ...

https://arxiv.org/html/2604.23338v2?utm_source=chatgpt.com